

ClassRoomSTORM Studio V1.2

User Help: Virtual and Experimental Super-Resolution Microscopy

Awanish Pratap Singh

Software version 1.2

Abstract

This Help document explains how to install and use ClassRoomSTORM Studio V1.2, a desktop teaching application for localization-based super-resolution microscopy. It describes the tasks you can perform and explains each screen and control. The physics behind the method (diffraction, blinking, localization, and precision) is explained in the companion document, *ClassRoomSTORM Theory Notes*.

Contents

1	What ClassRoomSTORM Is and Is Not	2
2	Concepts in 60 Seconds	2
3	Installation and Setup	3
4	Starting ClassRoomSTORM Studio	3
5	Quick Start: First Virtual Experiment and Reconstruction	4
6	Virtual Experiment Step by Step	5
6.1	Step 1: Pattern	5
6.2	Step 2: Blinking	5
6.3	Step 3: Image Formation	8
6.4	Step 4: Preview	8
6.5	Step 5: Export	9
7	SR Reconstruction Step by Step	9
7.1	Step 1: Load Video	9
7.2	Step 2: Calibration	9
7.3	Step 3: Preprocess	9
7.4	Step 4: Blinker Mode	10
7.5	Step 5: Processing	10
7.6	Step 6: Run and Results	11
8	Linked Profile Viewer	11
9	Reading Output Files and Diagnostic Figures	12
10	Input and Output File Table	14
11	Troubleshooting	14
12	Glossary	15
13	Instructor Notes	16

1 What ClassroomSTORM Is and Is Not

ClassroomSTORM Studio V1.2 is a desktop teaching application for *localization-based super-resolution microscopy*. It has two workflows:

- the **Virtual Experiment**, which creates synthetic “blinking” videos from a pattern of points; and
- **SR Reconstruction**, which turns a blinking video into a sharper *reconstructed image* by locating individual bright spots frame by frame and accumulating their positions.

ClassroomSTORM reproduces the *statistical and computational logic* of single-molecule localization microscopy (SMLM), the family of methods that includes STORM and PALM, in a form that needs no lasers, no fluorescence optics, and no scientific camera.

What this means

ClassroomSTORM is a *learning tool*, not a fluorescence microscope and not a research-grade analysis package. Its reconstruction deliberately uses a simple method that detects bright spots, calculates an intensity-weighted centroid, and accumulates the positions, so that students can inspect and understand every step. It does not perform Gaussian model fitting, maximum-likelihood estimation, or drift correction. Manual physical calibration is applied to exported coordinates; it does not estimate uncertainty (see the *Theory* document).

2 Concepts in 60 Seconds

Blinking video. A video with many frames in which, at any instant, only a few light sources (“blinkers”) are switched on. Each blinker appears as a small blurred spot.

Localize. In each frame, ClassroomSTORM finds the centre of every bright spot. Although the spot is blurry and several pixels wide, its *centre* can be estimated to better than one pixel.

Accumulate. The centres found in all frames are collected into a single image, the *reconstructed image*. Because it is built from sharp centres rather than blurry spots, it is much crisper than the raw video.

Compare. The arithmetic mean of the video frames (**Raw Mean**) is blurry; the **Super-Resolution** image is the accumulated reconstruction. Comparing the two is the central activity.

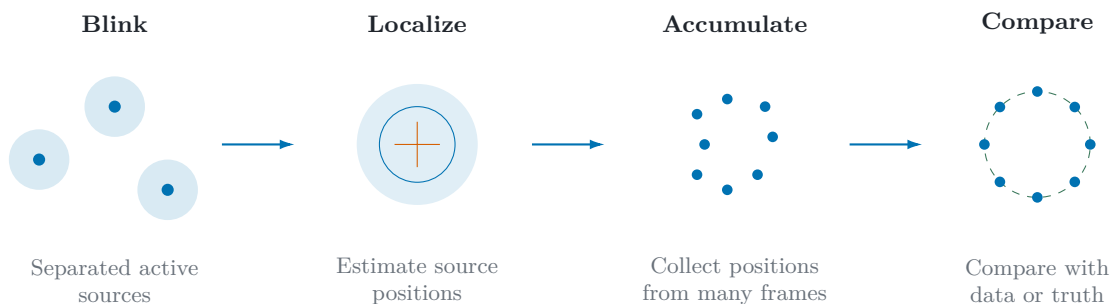


Figure 1: Conceptual progression from sparse blinking through position estimation to comparison with data or known truth.

The truth-data rule

When you create a Virtual Experiment, ClassroomSTORM also saves the exact positions it used, in a file called `truth.csv`. Reconstruction does *not* read this file. You compare your reconstruction against it only *afterwards*, to evaluate your result, much as you would check an answer key after solving a problem. Keep `truth.csv` in its original folder and do not feed it into the reconstruction.

3 Installation and Setup

ClassroomSTORM V1.2 is a Python desktop application. The user package includes ready-made setup and launch scripts, so most users never need to type any commands.

Requirements. The Windows setup script looks for Python 3.11, 3.12, or 3.13 automatically, using the Python launcher (`py`) when available. If no supported Python is found, it can offer to install Python 3.11 with `winget` or open the official Python download page. On macOS, install Python 3.10 or newer from <https://www.python.org/> before running setup. First-time setup downloads roughly 250-350 MB of packages from the Python Package Index (PyPI), so an internet connection is needed unless your package includes an offline `wheels` folder (see below).

Recommended: run the setup script.

- **Windows:** double-click `setup_windows.bat`.
- **macOS:** double-click `setup_macos.command`. The first time, macOS may block it; right-click the file and choose **Open**, or run `chmod +x setup_macos.command` in Terminal once.

The script builds a private environment (a `.venv` folder inside the package) and installs the required packages. When it finishes, it prints *Setup completed*. If it fails partway through, it preserves the environment for a retry. The launcher checks required imports before starting.

Offline installation. If the package folder contains a `wheels` folder, the setup script installs from it automatically and no internet access is needed. This is useful on restricted or proxy-protected networks.

Manual installation (advanced). Instead of using the script, you may open a terminal in the package folder and run `python -m venv .venv`, activate it (Windows PowerShell activation command shown below; macOS or Linux: `source .venv/bin/activate`), and then run `python -m pip install --upgrade pip` followed by `python -m pip install -r requirements.txt`. The standard packages are `numpy`, `opencv-python`, `matplotlib`, `Pillow`, and `PySide6`.

```
.\.venv\Scripts\Activate.ps1
```

Optional extras

Graphviz produces cleaner pipeline diagrams. Without it, ClassroomSTORM draws the same diagrams with Matplotlib instead, so Graphviz is not required. **NVIDIA CUDA GPU** acceleration is experimental; install it with `python -m pip install -r requirements-gpu.txt` only if you have a CUDA-capable NVIDIA card. AMD, Intel, and Apple Silicon GPUs are not supported in V1.2; those systems use the CPU.

4 Starting ClassroomSTORM Studio

Start the application with the launch script in the package folder:

- **Windows:** double-click `launch_studio_windows.bat`.
- **macOS:** double-click `launch_studio_macos.command`.

The launch script checks that the required packages are installed and, if they are missing, asks you to run the setup script first. Advanced users may instead run `python apps/ClassRoomSTORM_Studio.py` from an activated environment.

The ClassroomSTORM Studio launcher window offers two large buttons:

- **Virtual Experiment:** create blinking data from a logo, letter, or pattern (section 6);
- **SR Reconstruction:** reconstruct a recorded or simulated blinking video (section 7).

Two text links, **Help** and **Theory**, open this document and the Theory companion (the PDF files in the `docs` folder).

The launcher closes when you open a workflow

When you click **Virtual Experiment** or **SR Reconstruction**, the launcher window fades out and *closes itself*; the chosen workflow then opens as a separate window. This is normal; the application has not crashed. To return to the launcher, use the **Back to Studio** toolbar button at the top of the workflow window, or run the launch command again.

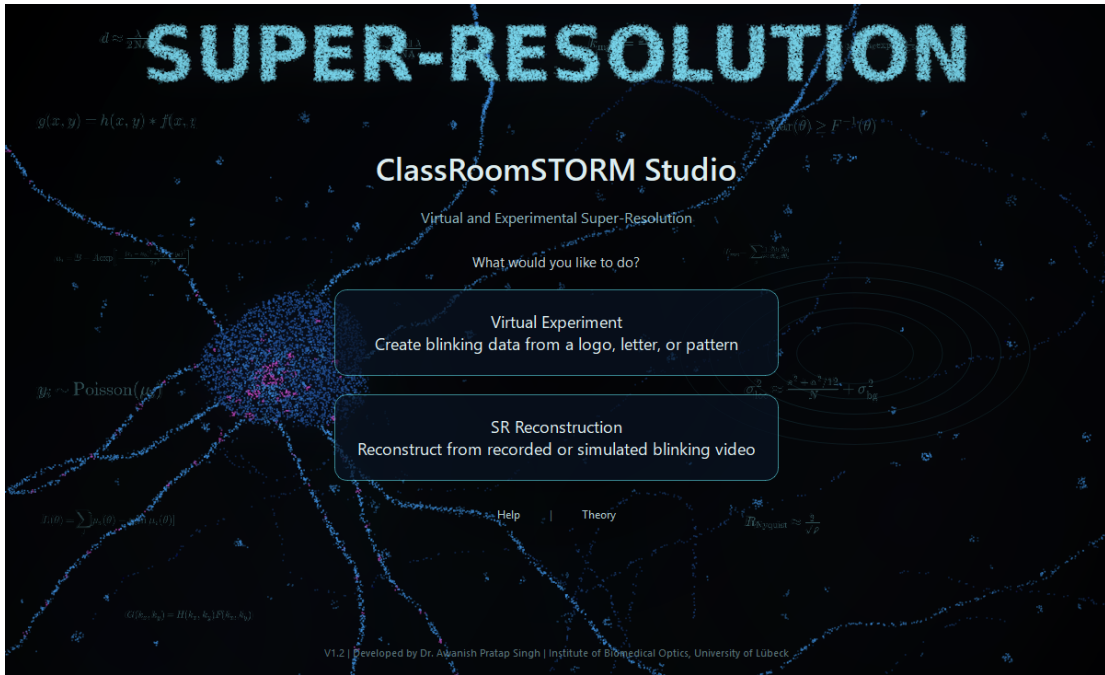


Figure 2: The ClassroomSTORM Studio launcher: the two workflow buttons (**Virtual Experiment**, **SR Reconstruction**) and the **Help** and **Theory** links, over the animated localization background.

5 Quick Start: First Virtual Experiment and Reconstruction

This walkthrough takes about ten minutes and uses only simulated data.

1. **Launch** the Studio and click **Virtual Experiment**.
2. On the **1 Pattern** page, set **Pattern** to **text** and type a short word (for example, **ABBE**) in the **Text** box.
3. On the **2 Blinking** page, set **Frames** to about 300 for a quick first run.

4. On the **3 Image Formation** page, keep the default settings.
5. On the **4 Preview** page, click **Preview Source Mask** and then **Preview Sample Blink Frame** to see what you are about to generate.
6. On the **5 Export** page, click **Export Dataset**. Note the output folder shown; it now contains `blinking_video.mp4`.
7. **Relaunch** the Studio (it closed in step 1) and click **SR Reconstruction**.
8. On the **1 Load Video** page, click **Browse Video** and select the `blinking_video.mp4` you just created. The first frame appears.
9. Leave **Blinker mode** on **auto** and **Processing backend** on **Serial**. On the **6 Results** page, click **Run Reconstruction**.
10. Open **Main Results**, then the **Linked Profiles** tab. Click on a bright part of the image and compare the wide red curve (raw) with the narrow blue curve (reconstruction).

6 Virtual Experiment Step by Step

The Virtual Experiment builds a synthetic blinking video. Its window has five numbered steps in the left-hand list: **1 Pattern**, **2 Blinking**, **3 Image Formation**, **4 Preview**, and **5 Export**.

6.1 Step 1: Pattern

The pattern defines *where* the source points are located. Choose one of three pattern types with the **Pattern** control.

Control	What it does	Default
Pattern	Source pattern type: <code>text</code> , <code>dots</code> , or <code>logo</code> .	<code>text</code>
Text	The word drawn in <code>text</code> mode, using a fixed 16×16 LED-style alphabet (A-Z, 0-9, space, -, _).	<code>ABBE</code>
Dot mode	For <code>dots</code> mode: <code>single</code> (one point), <code>pair</code> (two close points), or <code>random</code> (several points).	<code>single</code>
Logo	For <code>logo</code> mode: an image file (use Browse Logo). The image is converted to a black-and-white mask.	-
Logo scale	Scales the logo mask (0.1-5.0).	<code>1.0</code>
Logo sampling	How logo pixels become source points: <code>grid</code> , <code>all_pixels</code> , or <code>random_max_points</code> .	<code>grid</code>
Logo grid step px	Spacing of the sampling grid, in pixels (1-100).	<code>1</code>
Logo max source points	Upper limit on logo points; 0 means no limit.	<code>0</code>

Table 1: Controls on the **1 Pattern** page.

6.2 Step 2: Blinking

This page controls *how the source points switch on and off* over the frames of the video.

Why blinkers must be separated

If too many blinkers are on at once and they are close together, their blurred spots overlap and cannot be distinguished. Sparse, well-separated blinking is what makes localization possible; this is the core idea of the method (see the *Theory* document).

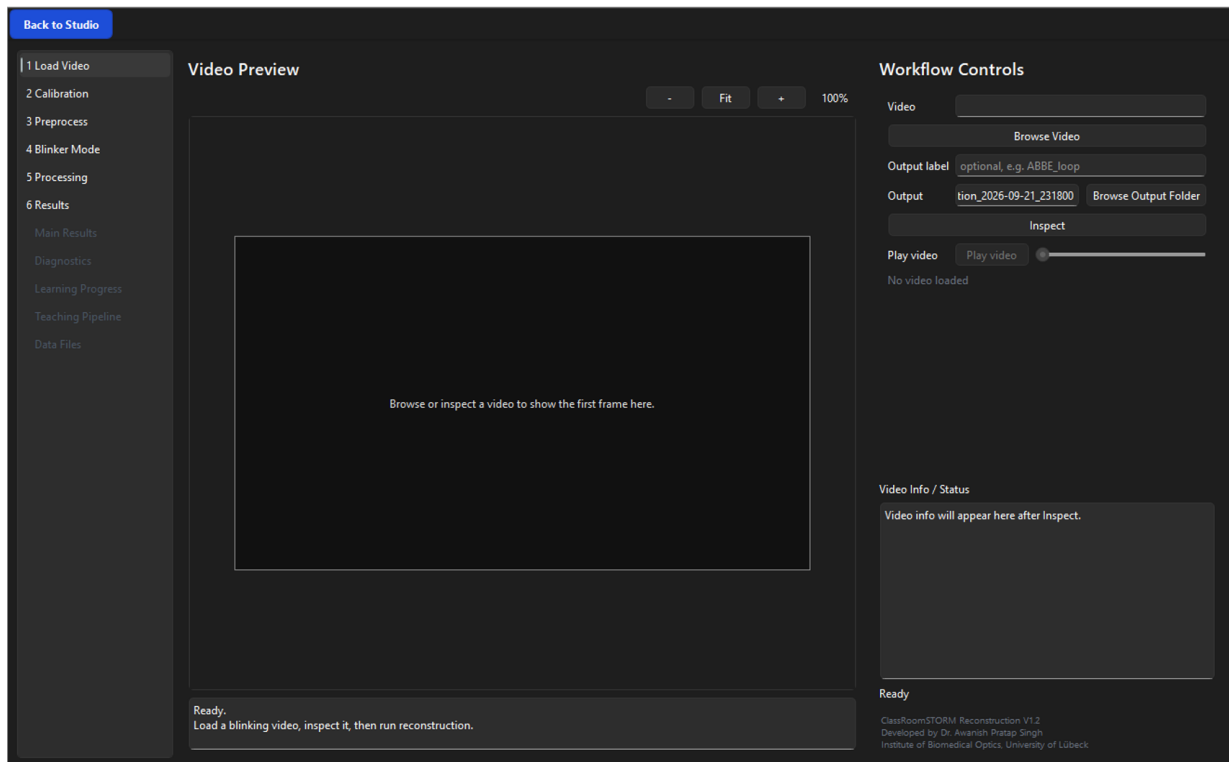
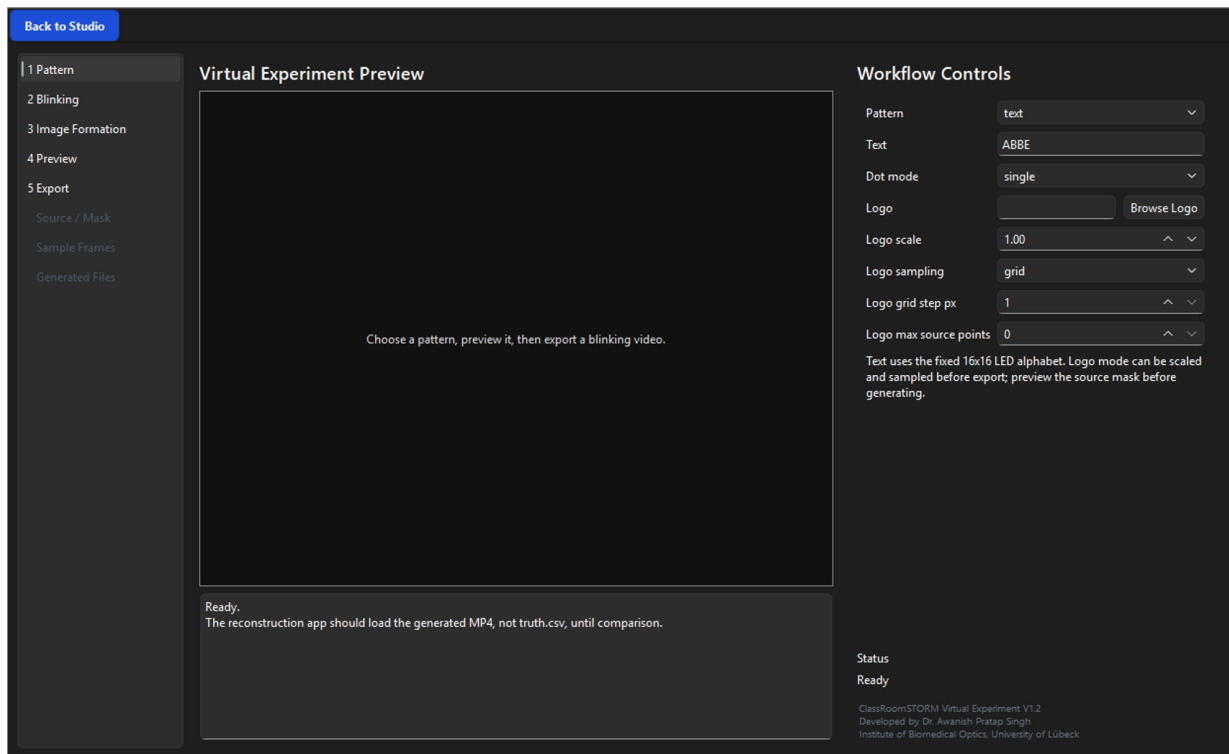


Figure 3: The two workflow windows used in the Quick Start: the Virtual Experiment (top) and SR Reconstruction (bottom).

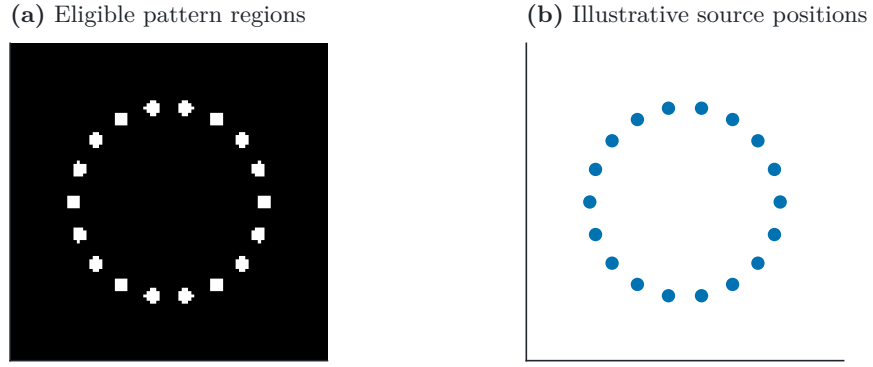


Figure 4: A schematic source pattern and one illustrative set of source positions. The mask defines eligible regions; it is not a measured optical image or a guarantee of one emitter per bright connected region.

Control	What it does	Default
Frames	Number of frames in the generated video (1-10000). More frames give a denser reconstruction but require more processing time.	1000
Max blinkers/frame	The largest number of source points allowed to be on in any one frame. The actual number may be lower if spacing rules cannot be met.	30
Blinker spacing	How far apart blinkers that are on simultaneously must be: separated, touching, overlapping, or random.	separated
Manual min distance px	A reference separation distance, in pixels, that the spacing mode scales.	100.0
Effective min distance	Read-only. Shows the separation actually applied.	-

Table 2: Controls on the **2 Blinking** page.

6.3 Step 3: Image Formation

This page controls the blur, brightness, and noise used to render each source point in a simulated camera image. Figure 5 shows one resulting frame.

Control	What it does	Default
sigma_px	The blur width σ of each spot, in pixels, or the point-spread width (0.5-50). Larger values give wider, blurrier spots.	10.0
Spacing px	The distance, in pixels, between neighbouring points of the source mask when they are placed in the image (1-100).	2.6
Brightness	The base number of signal photons per blinker (1-1 000 000).	80000
Effective brightness	Read-only. The brightness actually used (see the note below).	-
Background	A constant background level added to every pixel.	1.0
Read noise	The strength of the camera-like random read noise.	0.2

Table 3: Controls on the **3 Image Formation** page.

Brightness vs. Effective brightness

ClassRoomSTORM scales the **Brightness** you enter by a factor that grows with the number of blinkers per frame, so that frames with many blinkers remain visible. The scaled value is shown in the read-only **Effective brightness** field, and *that* value is used to render the frames. If your frames look brighter or dimmer than expected, check **Effective brightness**, not **Brightness**.

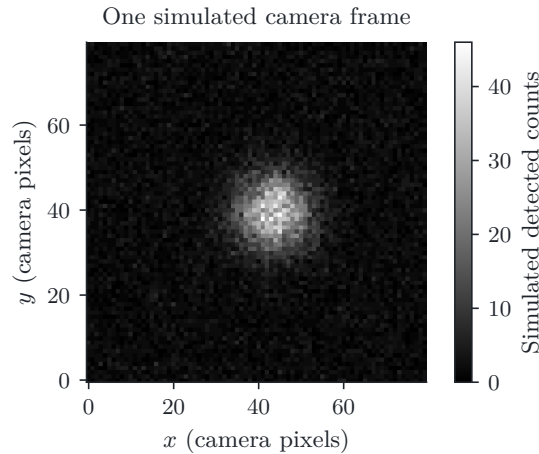


Figure 5: One raw synthetic frame from the recorded two-source run. Only one emitter is active in this frame; signal and background are Poisson-distributed.

6.4 Step 4: Preview

Two buttons let you check your settings before generating the dataset: **Preview Source Mask** shows the pattern of source points, and **Preview Sample Blink Frame** shows one example frame. Previewing does not write any files.

6.5 Step 5: Export

Choose an output folder (or accept the suggested one) and click **Export Dataset**. ClassroomSTORM writes the files listed in table 4 and then shows them in the result browser.

The **Frame backend** setting controls how virtual frames are rendered. **Auto CPU** is the recommended default. **CPU parallel** uses CPU workers, and **GPU CUDA** is experimental and requires PyTorch CUDA from `requirements-gpu.txt`. If CUDA is unavailable, ClassroomSTORM falls back to CPU generation and records the reason in `metadata.json`.

The **CPU workers (0=auto)** setting controls CPU frame rendering. Use 0 to let ClassroomSTORM use up to about 90% of available CPU workers, 1 for serial generation, or a larger value to limit CPU-parallel generation manually.

File	Contents
<code>blinking_video.mp4</code>	The blinking video, which is the input for reconstruction.
<code>truth.csv</code>	The exact source positions used. For comparison <i>after</i> reconstruction only (see the truth-data rule).
<code>preview_frame.png</code>	A sample blinking frame.
<code>pattern_mask.png</code>	The source-point mask.
<code>metadata.json</code>	All generation settings.

Table 4: Files written by the Virtual Experiment.

7 SR Reconstruction Step by Step

SR Reconstruction turns a blinking video into a reconstructed image. The video may be one you made with the Virtual Experiment or a recorded video of physical blinking sources. The window has six numbered steps: **1 Load Video**, **2 Calibration**, **3 Preprocess**, **4 Blinker Mode**, **5 Processing**, and **6 Results**.

7.1 Step 1: Load Video

Click **Browse Video** and choose an MP4, AVI, or MOV file. ClassroomSTORM reads the video and shows its first frame and basic information (size, frame count, frame rate). You may also set an **Output label** and an **Output** folder for the results. The **Inspect** button repeats this inspection at any time.

The **Play video** button plays the loaded video frame by frame in the preview, and the slider beside it scrubs through the frames, providing a quick way to see the raw blinking data before reconstructing it.

7.2 Step 2: Calibration

Calibration applies to exported coordinates

Choose **Manual calibration**, enter a positive **Unit per pixel**, and set the **Unit label**, for example, mm. The CSV retains crop-local `x_px/y_px`, adds original-frame `x_full_px/y_full_px`, and exports `x_calibrated/y_calibrated` relative to the original frame origin. Images and linked profiles retain pixel axes. Leave **Pixels only** selected when a physical scale is unavailable. Calibration does not establish precision.

7.3 Step 3: Preprocess

If you only want to reconstruct part of the field of view, tick **Enable crop** and set **Crop x**, **Crop y**, **Crop width**, and **Crop height**. A rectangle appears on the first-frame preview; **Show crop**

overlay and **Preview cropped region** let you check it.

Set the crop size before running

Crop width and **Crop height** start at 0. If you enable cropping but leave them at zero, reconstruction stops with an error. Inspecting a video fills these boxes with the full video size automatically; adjust them from there.

Background subtraction and threshold

Subtract frame median background subtracts the median intensity of each cropped frame and clips negative values to zero before thresholding and localization. This option is off by default and assumes sparse bright emitters on a roughly uniform background. The exported raw mean remains uncorrected. **Threshold quantile** defaults to 0.995. Blank and constant frames produce no localizations; invalid image arrays stop processing with an error.

7.4 Step 4: Blinker Mode

The **Blinker mode** control tells ClassroomSTORM how many bright spots to expect in each frame:

single At most one localization per frame, from the single brightest region.

multiple Several localizations per frame, with each separated bright region located independently.

auto ClassroomSTORM scans the first part of the video and chooses **single** or **multiple** for you. This is the default.

Tip

Use **single** for data with one blinker at a time, and **multiple** when several well-separated blinkers can be on together. If you are unsure, leave the mode on **auto**. The mode actually used is recorded in **summary.json**.

7.5 Step 5: Processing

The **Processing backend** controls how the computation is distributed.

Backend	What it does
Serial	Processes frames one at a time on the CPU. Simplest and most predictable; the recommended starting choice.
Auto CPU	Uses several CPU cores in parallel when that is worthwhile, capped at about 90% of available CPUs.
Only GPU	Uses an NVIDIA CUDA GPU for the single-blinker path.
Auto CPU+GPU	Uses the GPU for single blinkers and parallel CPU processing for multiple blinkers.
Auto CPU+GPU acceleration	An experimental hybrid path.

Table 5: Processing-backend options.

Workers (0=auto) sets the number of CPU processes; 0 lets ClassroomSTORM choose up to about 90% of available CPUs. **Max frames (0=all)** limits how many frames are processed, which is useful for a quick test run.

GPU backends fall back to the CPU silently

The GPU backends need an NVIDIA card with CUDA and the optional GPU packages installed. If the required hardware or packages are missing, ClassroomSTORM does *not* stop; it quietly runs on the CPU and records the fallback in `summary.json`. If GPU speed matters, open `summary.json` after the run and confirm that the GPU was used.

7.6 Step 6: Run and Results

Reconstruction and figure export run in a background worker, so the window remains responsive. Wait for completion before closing the application. Choose an empty output folder: existing results are protected from replacement. When truth comparison is selected, **Truth match radius (px)** sets the maximum allowed distance. Matching is one-to-one within the same frame and uses original-frame coordinates. Only ground-truth entries within the processed crop and frame range are evaluated. The summary reports precision, recall, F1, unmatched counts, and matched-position errors; undefined metrics are recorded as null.

On the **6 Results** page you may enter **Cumulative frames** (frame counts at which to save intermediate reconstructions, entered as comma-separated whole numbers) and tick **Compare with truth after run** if a `truth.csv` is available. Click **Run Reconstruction**. When reconstruction finishes, the results appear in the browser, grouped as **Main Results**, **Diagnostics**, **Learning Progress**, **Teaching Pipeline**, and **Data Files**.

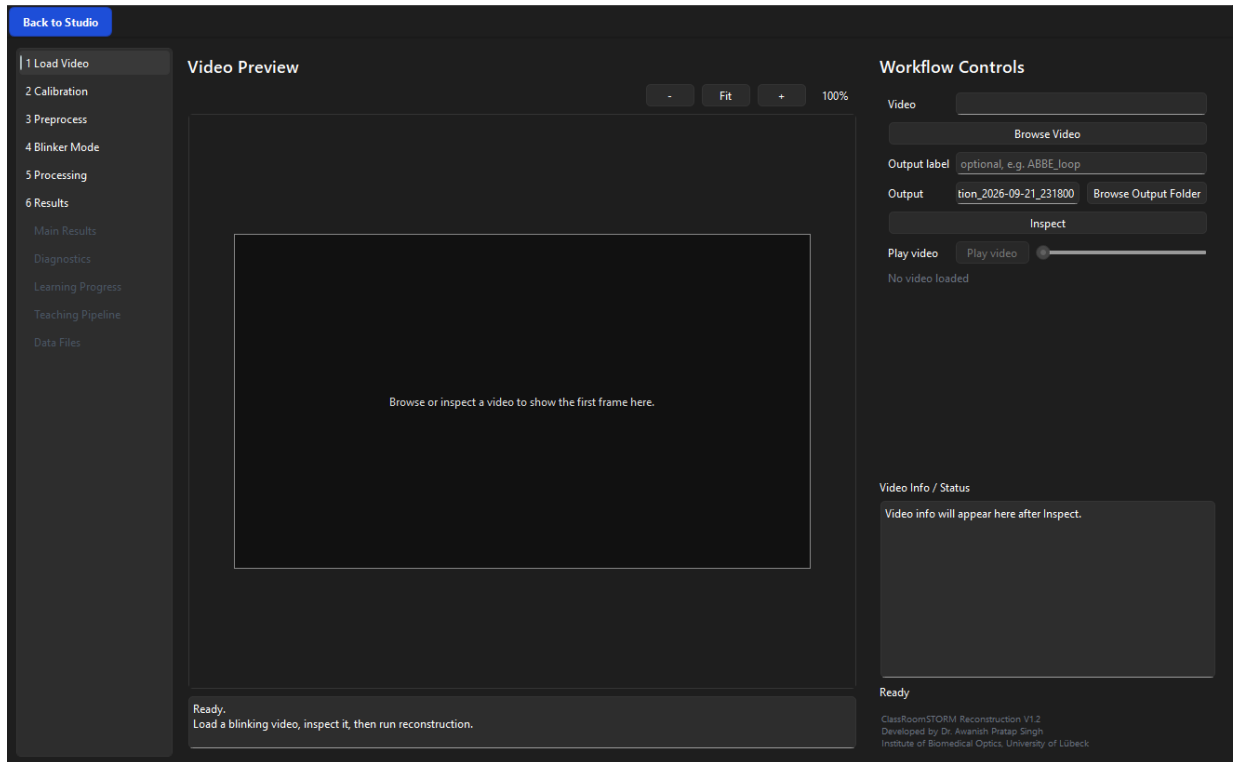


Figure 6: The SR Reconstruction window on the **1 Load Video** page, showing the numbered step list, the video preview, and the workflow controls.

8 Linked Profile Viewer

The **Linked Profiles** tab, in the **Main Results** group, is a way to compare the raw and reconstructed images. It appears when a reconstruction has produced both raw and reconstructed image data.

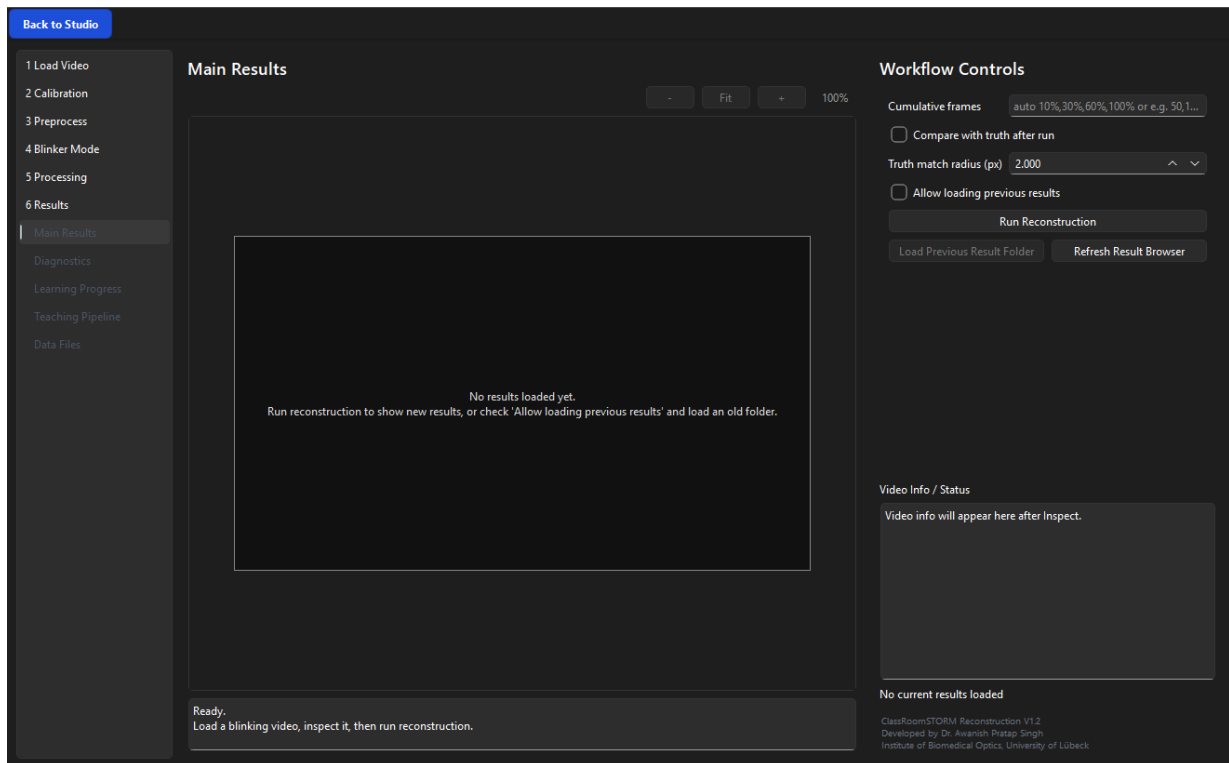


Figure 7: The SR Reconstruction **Main Results** page. The result browser fills with images, plots, and data files once a reconstruction has run.

The viewer shows the **Raw mean** image and the **Reconstruction** image side by side, with two line-profile plots beneath them. **Click anywhere on either image** to place a crosshair; the plots update to show the intensity along the row and the column through that point.

- The **red** curve is the **Raw mean** image, which appears as a wide, blurred bump.
- The **blue** curve is the **Reconstruction**, which displays positions using a fixed-width kernel.

The horizontal axis is the pixel coordinate; the vertical axis is normalized intensity (each curve is scaled to a 0-1 range so the shapes can be compared directly). Displayed peak width depends on the rendering kernel and normalization; a narrow reconstruction peak alone is not a resolution measurement.

9 Reading Output Files and Diagnostic Figures

Raw Mean The arithmetic mean of all processed frames, forming a diffraction-limited, blurry image.

Super-Resolution The reconstructed image, built by accumulating all localizations.

Raw vs Super-Resolution The two images side by side.

Localization Scatter / Density Map / Raw Overlay Diagnostic plots: localization positions, their density, and their distribution over the raw image.

Learning Progress Cumulative reconstructions showing how the image sharpens as more frames are added.

Teaching Pipeline A step-by-step report (`pipeline_report .html`) and pipeline diagrams.

Data Files The numerical results: `localizations.csv` and `summary.json`.

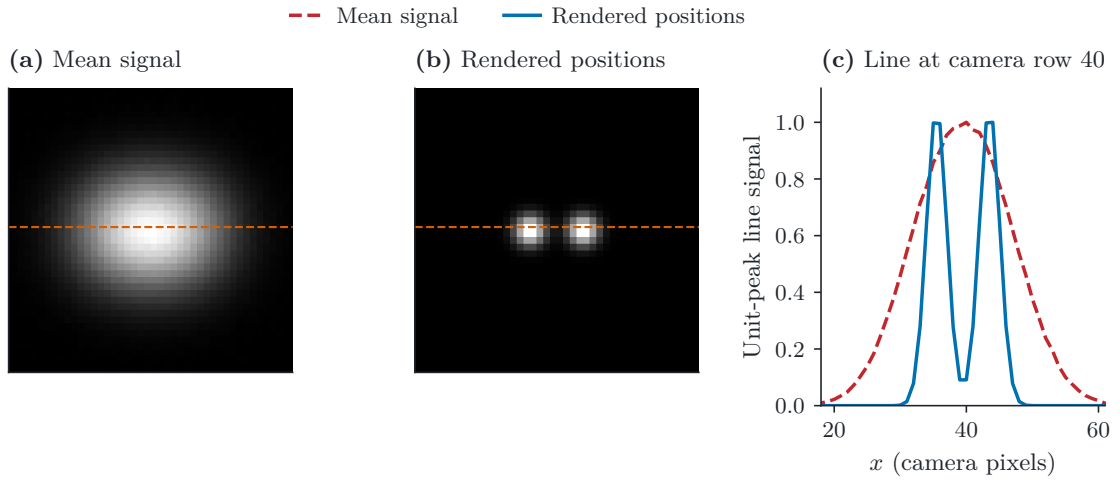


Figure 8: Linked row profiles from the recorded two-source synthetic reconstruction. Dashed horizontal lines identify camera row 40; each profile is peak-normalized. A profile describes the displayed images and is not an independent resolution measurement.

Virtual experiment	Reconstruction
<code>blinking_video.mp4</code> <code>truth.csv</code> : source positions <code>metadata.json</code> : settings <code>preview_frame.png</code> <code>pattern_mask.png</code>	<code>raw_mean.npy</code> , <code>superres.npy</code> <code>localizations.csv</code> : positions <code>summary.json</code> : run record Image and diagnostic exports Pipeline report and equations

Figure 9: Output roles for the virtual experiment and reconstruction workflows.

10 Input and Output File Table

Table 6 lists the files read and written by each workflow.

Workflow	Direction	Files
Virtual Experiment	reads	a logo image (logo mode only)
Virtual Experiment	writes	blinking_video.mp4, truth.csv, preview_frame.png, pattern_mask.png, metadata.json
SR Reconstruction	reads	a blinking video (MP4/AVI/MOV); optionally truth.csv for comparison
SR Reconstruction	writes	localizations.csv; raw_mean.npy and raw_mean.png; superres.npy and superres.png; raw_vs_superres.png; localization_scatter.png; plot_density_map.png; plot_raw_localization_overlay.png; plot_cumulative_reconstruction.png; summary.json; pipeline_report.md / .html; pipeline diagram files

Table 6: Files read and written by each workflow.

11 Troubleshooting

Setup fails to download the packages. This usually means that there is no internet access or that a restricted or proxy network prevents Python from verifying PyPI security certificates (a `CERTIFICATE_VERIFY_FAILED` error). Try an unrestricted network, ask your IT support how pip should reach PyPI, or use a package that includes an offline `wheels` folder.

Windows says Python was not found or opens the Microsoft Store. Run `setup_windows.bat` again and use its Python install option, or install Python 3.11, 3.12, or 3.13 from <https://www.python.org/>. If Windows keeps opening the Microsoft Store, disable the `python.exe` and `python3.exe` shortcuts in **Settings > Apps > Advanced app settings > App execution aliases**.

The application will not start, or a “ModuleNotFoundError” appears. The required packages are not installed. Run the setup script (`setup_windows.bat` or `setup_macos.command`) and wait for the *Setup completed* message before launching again.

The launcher disappeared after I clicked a workflow. This is expected. The launcher closes itself when a workflow opens. Use the **Back to Studio** toolbar button at the top of the workflow window, or run the launch command again to get it back.

No bright spots are detected / the reconstruction is empty. Check the contrast in the first video frame. Very dark or very noisy videos may have no clear spots. Try **multiple** blinker mode.

The reconstruction looks too sparse. Use more frames, or (for a Virtual Experiment) increase **Max blinkers/frame**.

A GPU backend seems no faster. It probably fell back to the CPU. Check `summary.json`; install the optional GPU packages if you have an NVIDIA CUDA card.

Reconstruction fails immediately with a crop error. **Crop width/Crop height** are zero. Inspect the video first to fill them, or untick **Enable crop**.

The video will not load. Make sure it is an MP4, AVI, or MOV file that OpenCV can read.

The Linked Profiles tab is missing. It appears only when a reconstruction has produced the raw and super-resolution data. Run a reconstruction first.

Truth comparison does nothing. Make sure `truth.csv` is in the same folder as the video and that **Compare with truth after run** is ticked.

A run is very slow. Lower **Max frames** for a test, or use **Auto CPU**.

12 Glossary

Blinker A light source that switches on and off; it appears as a bright spot when on.

Localization An estimate of the position of one bright spot's centre.

Centroid The brightness-weighted average position of a spot's pixels; the method Classroom-STORM uses to localize spots.

Threshold The brightness level above which pixels are treated as part of a spot.

Point-spread width (σ) How wide a single point of light appears after blurring; the `sigma_px` control.

Reconstructed (super-resolution) image The sharp image built by accumulating localizations.

Raw mean The blurry arithmetic mean of the video frames.

Ground truth The known, exact source positions, saved in `truth.csv`, used only to evaluate a reconstruction afterwards.

Frame One image in the video.

Signal photons The brightness of a blinker, set by **Brightness**.

Background A constant light level added to every pixel.

Read noise Random camera-like noise added to each pixel.

Field of view / crop The region of the video that is processed.

Pipeline report The step-by-step teaching report produced after a run.

13 Instructor Notes

Short session (2-3 hours). Use the Virtual Experiment only. Students generate a dataset, reconstruct it, and explore the linked profiles. One useful question is: *why* is the blue curve narrower than the red one?

Longer project (1-2 weeks). Add a recorded video of physical blinking sources (for example, from a classroom optical setup or an LED source) and have students compare reconstructions of simulated and real data.

Learning goals.

- Compare a raw (diffraction-limited) image with a localization-based reconstruction.
- Understand the truth-data rule: why the answer key is hidden until after reconstruction.
- Explore how blur width, noise, blinker density, and overlap affect the result.
- Treat physical calibration as optional; the core workflow uses pixel coordinates.

Suggested parameter sweeps. Have students vary one setting at a time and predict its effect before running the simulation: **sigma_px** (blur), **Read noise**, **Max blinkers/frame** (density and overlap), and **Frames** (sampling). The companion *Theory* document gives the physics behind each of these.